



DATA/MSML 603:
PRINCIPLES OF MACHINE LEARNING

FINAL PROJECT - FALL 2022
December 19, 2022

SpaceX Launch and Landing Data Analysis

Bishal Regmi

SCIENCE ACADEMY
UNIVERSITY OF MARYLAND, COLLEGE PARK

Contents

1. Research Question	3
2. Dataset	3
3. Data Cleanup and Exploration	3
4. ML Methodology	5
5. Results	6
6. Lessons Learned	9
A. Appendix	12
A.1. Google Colab Notebook	12

1. Research Question

This project, using the SpaceX launch data, will answer the following questions: What factors affect the success of the SpaceX rocket launches and recovery of the boosters? The end goal will be to determine, given factors such as payload size, mass, and other factors that would be controlled by a consumer, how successful would a rocket launch and recovery of the booster would be. As, the goal in to determine the success or failure of launch and recovery of the boosters, this project will focus on classification methodologies. This will include methods such as Decision Tree, K Nearest Neighbors, Support Vector Machines, and Random Forest.

Answering these questions is important because as society moves into a future where space becomes more accessible, there will be other options like Blue Origin and Virgin Galactic that will provide similar services. Understanding what factors affect the success of a launch, people can better decide which service provider and option to choose. Right now, due to lack of data from other services, this project will only be able to provide the consumers a level of confidence for launching with SpaceX. The level of confidence can be taken as the probability of success and failure.

2. Dataset

The dataset that will be used will be the one provided directly by SpaceX. SpaceX does a great job of providing data on their launches going into details on the rockets, launch, ships used to recover the boosters, payloads, launch pads, cores, etc. The main issue with this dataset is the number of observations. Due to these launches being fairly recent and scarce, the data set has around a total of 204 observations and additionally the data set is extremely class unbalanced as there are only four observation that were carrying a payload that failed. Sadly, given that other companies and agencies like NASA, Blue Origin, and Virgin Galactic do not provide any information or information even close to as detailed as what SpaceX provides, this will be the only source of data. The lack of other source of such data further enforces the importance of this dataset.

Going back to the data that is provided, the data is accessible via a REST API, with SpaceX providing a Postman collection to help users. The data is separate into different endpoints with each having information on separate topics. There are a vast number of variables in each endpoint where the data is stored as a nested JSON object. In order to get the data, the endpoints are accessed using Python Requests library and the JSON objects are massaged and put into a data frames. Overall, the data is semi-structured and not in a structured form like a table. The list of features that are used are separated into category for each endpoint and they are as follows: from launch endpoint, "id," "date utc," "launchpad," "rocket," "payloads," "cores," and "success" are used. From launch pad endpoint, "name," "latitude," and "longitude" are used. From payload endpoint, "mass kg" and "orbit" are used. From rockets endpoint, "name," "stages," "boosters," "cost per launch," "success rate pct," and "payload weights" are used. From core endpoint, "flight," "gridfins," "legs," "reused," "landing attempt," "landing success" are used. Some of these variables are ignored when creating the model. The final list of variables includes the following: "launchpad name," "mass kg," "orbit," "rocket name," "rocket success rate pct," "flight," "gridfins," "legs," "reused," "landing attempt," "landing success," and "success." The target variables are launch success from the launch endpoint and landing success variable from the core endpoint. Each target variable is used as the target for separate models.

3. Data Cleanup and Exploration

Even though the data is provided from an API directly from SpaceX, the dataset is anything but clean or ready to use. There are a lot of missing values, and the modality of the data is

problematic to say the least. The nested JSON object structure makes it difficult to directly place the data into a table as it requires flattening the object to first get it into a table form. Additionally, this requires careful examination of the features provided and removing any that are unnecessary before trying to flatten the object.

The launch data endpoint is the first to be hit. Following the GET request to the endpoint and conversion to JSON object, the dataset is filtered to only contain the columns stated above. There are a few observations where the values are missing in the endpoint but publicly available. These values are then manually adjusted and fixed. Once, manually fixed, the next step is to remove the observations where the target value is null. This drops the total number of observations down from 204 to 191.

Once the launch data is cleaned up, the launch id and the specific columns containing keys to access data from other endpoints are used to connect all the other information together. The second endpoint that is hit is the launchpad endpoint to pull three columns as listed above. The launch id is also kept together with the observations to later merge all the data together. An interactive map is also created with the data to plot icon on the launchpad locations and the number of launches from each site. This is available to access using the link to the Google Colab notebook in Appendix. There are three launch sites within the US and one outside. Most of launches happened at CCSFS SLC 40 with 102 launches. This launch site is in Florida.

The next endpoint is payload. Again, similar to how the launchpad data is extracted, the key for the payloads from the launch data is used to query the information for each payload. The launch id along with the data of the payload for from the specific columns listed above are stored together. As some launches had multiple payloads and as this process splits apart the payload to separate rows for each payload, this leads to increase in the number of observations when merging back to the launch data. Also, as some launches did not have any payload, those observations end of removed from the dataset. The total amount of observation is not at 191 again. The analysis for this data consists of checking the number of payloads sent to each orbit and the different orbits and the total mass sent to each. The orbit with the most payload sent is VLEO with GTO at second and LEO at third. The orbit with the most amount of mass sent is VLEO at first, then GTO, and then PO at third. Further details on this analysis are again available to access using the link to the Google Colab notebook in Appendix. The rockets endpoint is the next endpoint. The desired columns are pulled from the endpoint and again the launch id is kept together with the observations to later merge with launch data. The main analysis for the data from this endpoint is the check for the three unique rockets that are used in the launches and their stats along with the amount of time each rocket was used. Falcon 9 is the most used rocket and the Falcon 1 is the second most used far lower than Falcon 9. Falcon 1 is also the main rocket to fail the most amount of time with a payload. Further details on this analysis are again available to access using the link to the Google Colab notebook in Appendix.

The last endpoint is the core endpoint which follows the same step as the other endpoint hits. The main difference with this endpoint is that a manual change was added to the “landing attempt” and “landing success” column. If the data points were “NONE” then they were set to false as this was the case for most of the missing data. There was no further analysis done with this endpoint.

Once the data has been pulled from all the endpoints, the final step is to merge it using the launch id using inner join to pull all the data together. The extra columns created in the merge process along with all the columns with keys are then removed. Then, all the rows with any null values are removed. Once all of this is done, the data is given a final glance to further analyze it and then the dataframe is reduced down to the list of final variables listed above. The correlation between each column is checked to remove any variables that highly correlated. Following the correlation analysis. The last step in the cleanup process is converting all the Boolean columns to integers. The main issue with the data is finally clear with there being only four failed launches. The data is highly unbalanced for creating a model to determine if a launch will be successful.

Now, the data is ready for splitting and scaling. In order to make sure that each split has each of the classes, random stratified sampling is done with 25 percent of the data in test and 75 percent in train. For success, this places one of the four fails in test and three in train. This heavily affects how the grid search is done. Long story short, most of the models are not going to be able to detect the fails at all. This is not so much of an issue with the landing success. The data is split and X and Y variables for each test and train are then set up. This process is done for both launch success and the landing success models. Before the splits, the X variables for both cases are scaled using the Sklearn “StandardScaler” function. Now the data is finally ready to be used in the models for test and train.

4. ML Methodology

Going in line with the goal of the project, this project will use four different classification methods. The list of methods is as follows: Decision Tree, K Nearest Neighbors, Support Vector Machines, and Random Forest. These methods were chosen as they are great for classification problem with their variety of strengths and weaknesses. The Random Forest is for the feature importance analysis which answers the key question for this project. Also, as these methods are classification method, various metrics for measuring the performance of the classification models are used. The main method of visualizing the model’s performance is a confusion matrix. This allows for the calculation of values such as accuracy, precision, recall or sensitivity, and additionally the F1-score. The goal was to plot a ROC curve and have an AUC measure as well. This plan falls apart when classes are extremely unbalanced as with the case of this dataset. The AUC measures and even values such as precision and recall require that the target be manipulated so that these metrics can be used for the grid search algorithms. This manipulation involves swapping the values of 1 and 0 in the target columns so that recall can be used to measure if the model can accurately catch all the failed launches.

Decision Tree is a supervised learning, non-parametric algorithm which allows for both classification and regression. It is used for classification in this project. It is a greedy algorithm that tries to find ways to best split the data using metrics such as entropy/information gain or gini impurity. The algorithm greedily finds the best splits and turns the data into a tree form with a starting root node which branches into other multiple nodes and leaves as the bottom. This visual is present in the notebook in Appendix as well. For this algorithm, there are multiple hyperparameters that can be tuned. For this project the following hyperparameters will be turned. The “criterion” parameter will be changed between “gini” and “entropy.” The “splitter” parameter will be changed between “best” and “random.” The “max depth” parameter will have value ranging from 1 to 9. The “max features” parameter will be changed between “auto” and “sqrt.” The “min sample leaf” parameter will have values 1, 2, or 4. The “min sample split” parameter will have values 2, 4, or 6. All these parameters affect how the splitting is done with either change in the metric used or how the tree is formed with the number of splits and the total number of layers. The direct effect on performance is visible with the recall and accuracy score in the notebook provided in the Appendix.

The different parameters are passed into the GridSearchCV function along with the classifier using 3-folds cross validation for launch success and 10-fold cross validation for landing success. The low number of folds in the launch success classification is to account for the fact there are only three failed events in the training and the metric recall which is used in the grid search best parameter selection process needs the fails to be in each split. Additionally, the grid search function is scoring the parameters with recall and accuracy with recall being used to refit the model. As stated earlier, the scores for each of the combination of parameters are available in the Colab notebook in the appendix.

The next algorithm K Nearest Neighbor or KNN is yet another nonparametric supervised learning classifier which different metric to measure proximity to classify the group that each

observation falls into. The metric used in KNN are usually different distance or closeness measures. This can also be used for both regression and classification, but it is used for classification for this project. The great thing about KNN is the few hyperparameters it has to tune. This makes the algorithm easy to use. In this project, the following hyperparameters will be tuned. As minkowski is the default metric of measure, the p value is changed between 1 and 2 to adjust between L1 and L2 distances. The “n neighbors” value is set between 1 to 10. The best values given a binary classifier should be less than the total number of fails as that class has less than 4 instances in the dataset. The last parameter that is changed is the algorithm parameter which is set to the following values: “auto,” “ball tree,” “kd tree,” and “brute.” All of these parameters affect how the clusters are made. Again, the direct effect on performance is visible in the Colab notebook in the Appendix. The grid search function is set up similar to how it is set up for the decision tree algorithm.

Support Vector Machine or SVM is another supervised learning algorithm that is used to both classification and regression. The goal of the algorithm is to create a decision boundary that splits the observation into the distinct number of groups that the observations are broken into. The decision boundary is usually linear by default and can be changed to nonlinear by using kernels. SVM has many parameters but for this project only a few are tuned. The kernel parameter is tuned with “rbf” and “sigmoid.” The “c” parameter used for regularization which is inversely proportional to the strength of the regularization is tuned with values 0.1, 1, 10, 100, and 1000. The “gamma” parameter which is a kernel coefficient for both of the kernels used is set to values such as 1, 0.1, 0.01, 0.001, and 0.0001. The grid search function is set up similar to how it is done for the previous for models as well.

Random Forest is the last algorithm, and it is yet another supervised learning algorithm that can be used for both classification and regression. It utilizes ensemble learning technique, bagging, to help its performance. Instead of creating only one tree like in decision tree. It creates multiple trees and uses the decision of all the trees to make the final decision. It has parameters that are similar to that of decision tree and more. For this project the following parameters are tuned. The “n estimators” parameter is tuned to values 100 or 200. The “criterion,” “max depth,” “min sample leaf,” and “min sample split” parameters are tuned similar to how it is tuned in decision tree. Most other parameters would be turned but the time complexity of this algorithms is far greater than that of the decision tree and with addition of multiple parameters the grid search function takes far too much time. These other parameters include “max features” and “oob score.” The grid search function for random forest is also setup similar to how all other models were set up. The results and effect of the different parameters on the performance is directly shown in the notebook in the Appendix.

5. Results

Once all the models are setup with grid search, they are trained and tested. The best parameters that the grid search finds are used to test the different model. Both the scores for train and test are printed in the notebook but this analysis is going over only the test data analysis with a short summary for train. The feature importance plot extracted from random forest is also discussed along with the plot created from the feature importance data.

```

DT launch -----
tuned hyperparameters :(best parameters) {'criterion': 'gini', 'max_depth': 1, 'max_features': 'auto', 'min_samples_leaf': 1, 'min_samples_split': 2, 'splitter': 'best'}
[[56 0]
 [ 1 0]]
      precision    recall  f1-score   support

     0       0.98        1.00        0.99         56
     1       0.00        0.00        0.00          1

   accuracy          0.98         57
  macro avg       0.49        0.50        0.50         57
 weighted avg       0.97        0.98        0.97         57

KNN launch -----
tuned hyperparameters :(best parameters) {'algorithm': 'auto', 'n_neighbors': 1, 'p': 1}
[[55 1]
 [ 1 0]]
      precision    recall  f1-score   support

     0       0.98        0.98        0.98         56
     1       0.00        0.00        0.00          1

   accuracy          0.96         57
  macro avg       0.49        0.49        0.49         57
 weighted avg       0.96        0.96        0.96         57

SVM launch -----
tuned hyperparameters :(best parameters) {'C': 0.1, 'gamma': 1, 'kernel': 'rbf'}
[[56 0]
 [ 1 0]]
      precision    recall  f1-score   support

     0       0.98        1.00        0.99         56
     1       0.00        0.00        0.00          1

   accuracy          0.98         57
  macro avg       0.49        0.50        0.50         57
 weighted avg       0.97        0.98        0.97         57

RF launch -----
tuned hyperparameters :(best parameters) {'criterion': 'gini', 'max_depth': 1, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100, 'n_jobs': -1}
[[56 0]
 [ 1 0]]
      precision    recall  f1-score   support

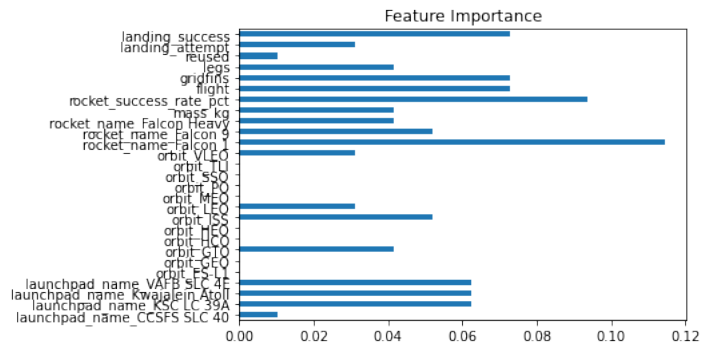
     0       0.98        1.00        0.99         56
     1       0.00        0.00        0.00          1

   accuracy          0.98         57
  macro avg       0.49        0.50        0.50         57
 weighted avg       0.97        0.98        0.97         57

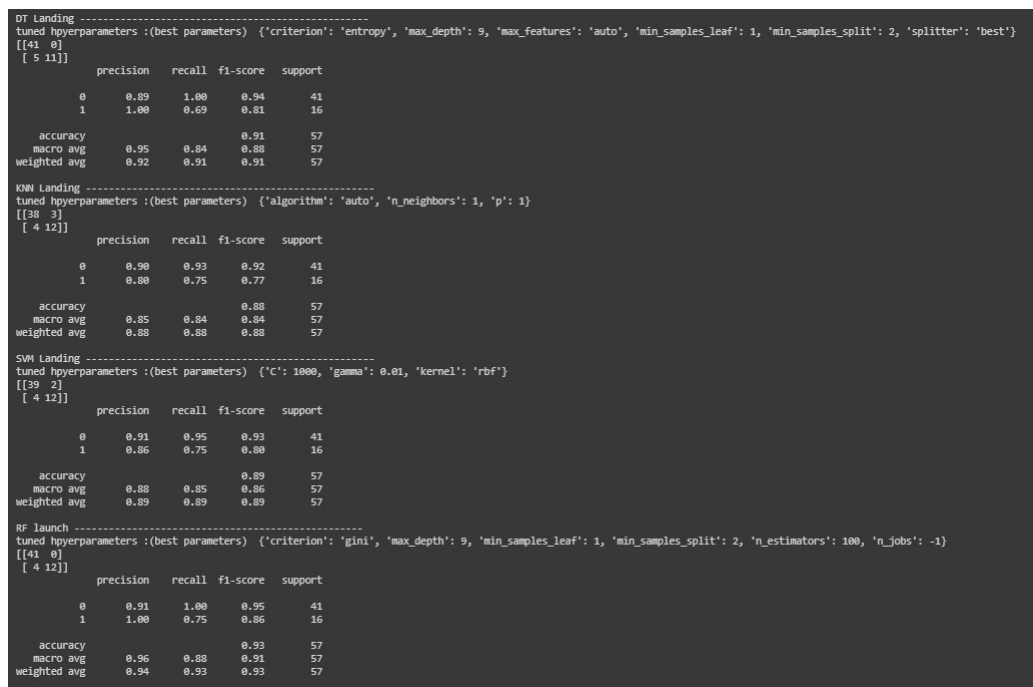
```

Figure 1: Model Performance with Launch Data

Starting off with the launch success classification and all the results are presented in Figure 1 above. The first model is decision tree. The best parameters that are selected by grid search are gini criterion, max depth of 1, max features set to auto, min samples leaf set to 1, min sample split set to 2, and splitter set to best. As visible in the figure, the model did a good job with successful launches but does horrible when it comes to fails. The confusion matrix shows that all 57 launches in the test set are considered successful when one was a failed launch, this model is not good given that that the most important class is the failed class but due to the rarity of the event in the observation, the model did not do well. The recall score of zero for class 1 which is failed launch is at zero. Even the train model had similar issues. Additionally, even though the accuracy is 0.98 this model will not catch if a launch will fail properly. This issue is persisted for all models. Even the most robust random forest fails to catch the failed launch. The only model that caught the failed launch was the KNN in the training set, but it was only overfitting as it failed to do so in the test set. Additionally, the KNN model failed to capture all the successful launches as well where it considered a successful launch a failure. The best parameters for the KNN are algorithm set to auto and k value set to 1 and p value set to 1. The best parameters for SVM are c set to 0.1, gamma set to 1, and kernel set to rbf. Finally for random forest, the best parameters are gini criterion, max depth of 1, min samples leaf set to 1, min sample split set to 2, and “n estimators” set to 100. Overall, none of the models were able to capture the failure properly.



In Figure 2, the feature importance that was calculated by the random forest for the launch success is displayed. According to the plot, the most important feature is the rocket name being Falcon 1. From prior research this makes sense as most of Falcon 1 launches failed so knowing if this rocket is used or not is important. Following the rocket used, the success rate of the rocket is the next most important feature. This again makes sense as the success rate should be strongly related to how well the rocket performs again. It is interesting to see that when checking correlation, the success percentage of rocket was not strongly correlated to the success rate, but this shows that it does matter as expected.



Moving on to the land success classification. All the results are presented in Figure 2 above. The first model is again the decision tree. The best parameters that are selected for decision tree are entropy criterion, max depth of 9, max features set to auto, min samples leaf set to 1, min sample split set to 2, and splitter set to best. The max depth of 9 is concerning and the max depth values provided should have been increased. A point to further improve on this project. The best parameters for the KNN are algorithm set to auto and k value set to 1 and p value set

to 1. The best parameters for SVM are c set to 1000, γ set to 0.01, and kernel set to rbf. Finally for random forest, the best parameters are gini criterion, max depth of 9, min samples leaf set to 1, min sample split set to 2, and “n estimators” set to 100. Again, the max depth of 9 is concerning. The performance of the models for landing classification is far better than that of the launch models. The decision tree had a recall of 0.69 and an accuracy of 0.91 which is great. KNN improves upon the recall score with a score of 0.75 for fails but the accuracy falls to 0.88. The decision tree did a much better job when looking at f1-scores. SVM again does not do as well as the decision tree either with both f1-scores being lower than that of decision tree. The random forest did better than all the other models with a f1 score of 0.95 for success and 0.86 for fail. Additionally, the model is still getting a recall of 1 for success and 0.75 for fails. Again, if the max depth could have had a great range than up to 9, there is possibility that the performance would be even better. This is again a point to further improve on this project.

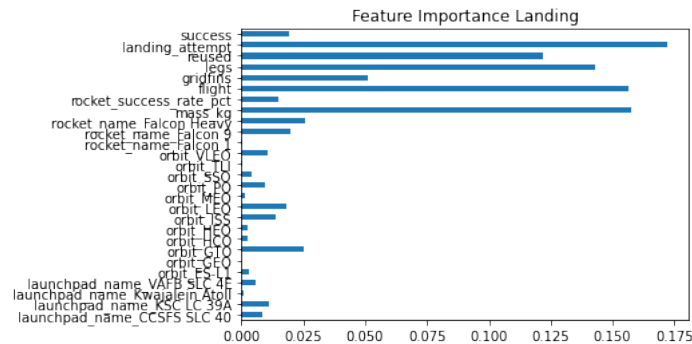


Figure 4: Land Data Feature Importance

In Figure 4, the feature importance that was calculated by the random forest for the landing success is displayed. According to the plot, the most important feature is the landing attempt, then mass of the payload and flight number followed by legs and reused. These top five features all make sense as the landing attempt heavily affects if the boosters land or not. Additionally, the flight number tells how many times this core has already flown and landed successfully. If there are no legs then the boosters can not land. If the boosters are reused then it landed before and most likely will land again. The most interesting feature is the payload feature which was the second most important. Not sure how the payload is directly connected. This will require further research into the data to see how the landing success variable looks with the different payload mass. This would be another great point of further research in the future.

6. Lessons Learned

There are many lessons to take away from this project. As discussed in the results, there are a few points of issues where the models could be improved. Especially with the decision tree and the random forest with the landing data where the max depth could have been higher. Other than that, the main thing that can be learned from this project is the importance of class balance and how difficult creating a model can be when a class is rare. With the launch data, all the models do not do well. The main goal of this project is to be able to detect when a SpaceX launch or landing might fail. The models for landing might be fine, but the models for launch are not good at detecting launches that could fail. If there was more data this could be improved but given the situation, that is not possible. With that improving the launch models will not be an easy task. Going back to the main question for the projects there are variety of factors that affect both launch and landing. But the top 5 key features for detecting launch fail are the rocket being used being Falcon 1, the rocket success rate, the core having gridfins, the number of

times the core has flown, and if the landing is a success. It is still possible that the launch fails due to payload not being sent to the right orbit, but the boosters can still land back. Therefore, the last feature is kept. For the landing, the top 5 key features that affect launch fail are landing attempted, mass of payload, flight number, if the core has legs, and if the core is reused.

References

- [1] <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html/>
- [2] https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html/
- [3] <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html/>
- [4] <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html/>
- [5] <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- [8] https://scikit-learn.org/stable/modules/model_evaluation.html#scoring-parameter/

A. Appendix

A.1. Google Colab Notebook

Note: The graphs and information on the notebook might look different as the random states were not set. The plots in the .ipynb document is also the same as the one online. I forgot to download before I accidentally re-ran the whole notebook.

https://colab.research.google.com/drive/168msBHtGGTjvbQr23r_0U36BSqt0IG_1?usp=sharing